

ABSTRACT

The rapid growth of digital communication, email has become one of the most important methods for sharing information. Millions of emails are sent every day for personal, educational, business, and professional purposes. However, along with useful emails, a large number of unwanted and harmful emails, known as spam emails, are also sent to users. Spam emails may contain advertisements, fake offers, phishing links, fraudulent messages, malicious content, or unnecessary promotional information. These emails can waste users' time, reduce productivity, and sometimes create serious security and privacy problems. Therefore, automatically identifying and filtering spam emails has become an important application of Machine Learning (ML) and Natural Language Processing (NLP).

The main aim of this project is to build and compare different Machine Learning classification models for spam email detection. The project uses a text-based spam dataset containing email or message content along with their corresponding labels, such as spam and ham (not spam). The dataset is analyzed and processed before applying the Machine Learning algorithms. Since Machine Learning algorithms cannot directly understand raw text, the text data must first be converted into a numerical format. This project follows a systematic process that includes dataset exploration, data preprocessing, text vectorization, dimensionality reduction, model training, evaluation, and performance comparison. The first stage of the project is dataset loading and exploration.

The dataset is loaded into Python using the Pandas library. Different properties of the dataset are examined, including the number of rows and columns, column names, data types, missing values, duplicate values, and the distribution of spam and ham messages. Exploring the dataset helps in understanding the nature and quality of the available data. This step is important because the performance of Machine Learning models depends greatly on the quality of the input data. Data visualization techniques are also used to present important information in graphical form. Visualizations make it easier to understand the dataset and observe patterns in the distribution of different classes.

The next important stage is data preprocessing. Real-world datasets often contain missing values, duplicate records, unnecessary columns, and inconsistent data. These problems can negatively affect the performance of Machine Learning models. Therefore, preprocessing is performed to clean and prepare the dataset for analysis. Unnecessary columns are removed, missing values are handled if present, duplicate records may be removed, and the target labels are converted into a suitable numerical format. The text data is also prepared for feature extraction. Data preprocessing improves the quality of the dataset and helps the algorithms learn useful patterns more effectively.

INTRODUCTION

In today's digital world, communication through emails and messages has become an important part of daily life. People use emails for personal communication, education, business, banking, online shopping, and many other purposes. Every day, millions of emails and messages are exchanged across the world. While many of these messages contain useful and important information, a large number of unwanted messages are also sent to users. These unwanted messages are commonly known as spam.

Spam messages may contain advertisements, promotional offers, fake prizes, fraudulent links, phishing attempts, or other unwanted content. Some spam messages are simply annoying, while others can be dangerous. For example, a phishing email may try to steal personal information such as passwords, bank details, or account information. Because of the large number of emails received by users every day, manually checking and identifying spam messages is difficult and time-consuming. Therefore, an automatic system for detecting spam messages is necessary.

Spam detection is the process of identifying whether a message is spam or legitimate. A legitimate message is often called ham in spam classification problems. The main objective of a spam detection system is to correctly classify incoming messages into two categories:

Spam: Unwanted or harmful messages.

Ham: Genuine and legitimate messages.

In earlier systems, spam filtering was mainly based on manually created rules. For example, certain suspicious words such as "free," "winner," "cash," or "offer" could be used to identify spam messages. However, rule-based systems have limitations because spammers can easily change the words and format of their messages. A manually created rule may also incorrectly classify genuine messages as spam. Therefore, more intelligent and automatic methods are required.

Spam detection is also closely related to Natural Language Processing (NLP). NLP is a field of Artificial Intelligence that helps computers understand, process, and analyze human language. Since emails and messages contain text, NLP techniques are required to convert the text into a form that Machine Learning algorithms can understand. A computer cannot directly use words and sentences as input for most Machine Learning algorithms. Therefore, the text must be converted into numerical values or features.

One of the important techniques used in this project is TF-IDF (Term Frequency-Inverse Document Frequency). TF-IDF converts text into numerical features based on the importance of words. Words that are useful for distinguishing between spam and ham messages receive higher importance. This process helps

OBJECTIVE

The main objective of this project is to develop and evaluate a Machine Learning-based spam message classification system using Natural Language Processing techniques. The system is designed to automatically identify whether a given message belongs to the spam category or the ham (legitimate) category. Spam messages are unwanted messages that may contain advertisements, fake offers, phishing links, fraudulent information, or other unnecessary content. Ham messages are genuine and useful messages sent for normal communication. Automatically identifying spam messages is important because manually checking a large number of emails or messages is difficult, time-consuming, and inefficient.

The main objective of this project is to develop a Machine Learning-based Spam Message Classification System using Natural Language Processing (NLP) techniques. The key objectives are:

- To classify text messages into Spam and Ham (legitimate) categories.
- To explore, clean, and preprocess the SMS Spam dataset before model training.
- To convert text messages into numerical features using TF-IDF Vectorization.
- To apply SVD (Singular Value Decomposition) for dimensionality reduction.
- To implement five Machine Learning algorithms: Logistic Regression, Extra Trees Classifier, LDA, QDA, and MLP Classifier.
- To evaluate each model using Accuracy, Precision, Recall, and F1-Score.
- To compare the performance of all five algorithms and identify the best-performing model for spam detection.
- To understand the complete Machine Learning and NLP workflow, from data preprocessing to model evaluation.
- To convert text into numerical features using TF-IDF (Term Frequency-Inverse Document Frequency) vectorization.
- To reduce the dimensionality of text features using Singular Value Decomposition (SVD) while retaining important information.
- To divide the dataset into training and testing sets for reliable model evaluation.
- To implement Logistic Regression for binary spam classification.
- To implement Extra Trees Classifier and study its performance as an ensemble learning method.
- To apply Linear Discriminant Analysis (LDA) for separating spam and ham classes.
- To apply Quadratic Discriminant Analysis (QDA) to study classification using flexible decision boundaries.

METHODOLOGY

This project follows a systematic Machine Learning methodology to develop a spam message classification system. The methodology includes importing the required libraries, loading the dataset, dividing the data into training and testing sets, converting text into numerical features using TF-IDF, reducing the dimensionality using Singular Value Decomposition (SVD), and evaluating the performance of the Machine Learning models using suitable evaluation metrics.

The main purpose of following this methodology is to transform the raw text data into a suitable format that can be understood by Machine Learning algorithms. Each step plays an important role in improving the efficiency and performance of the classification models.

Import Libraries

The first step in the project is importing the required Python libraries. Python provides many powerful libraries that make data analysis, visualization, preprocessing, Machine Learning, and model evaluation easier.

The important libraries used in this project include:

NumPy: Used for numerical calculations and array operations.

Pandas: Used for loading, organizing, and analyzing the dataset.

Matplotlib: Used for creating graphs and visualizations.

Seaborn: Used for creating attractive statistical visualizations.

Scikit learn: Used for Machine Learning algorithms, preprocessing, train-test splitting, TF-IDF vectorization, SVD, and evaluation metrics.

The required Machine Learning models are also imported from the Scikit-learn library. These include Logistic Regression, Extra Trees Classifier, Linear Discriminant Analysis, Quadratic Discriminant Analysis, and MLP Classifier.

Importing libraries at the beginning of the program makes the required functions and algorithms available for use throughout the project.

Example code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Different modules from Scikit-learn are imported for preprocessing, Machine Learning, and evaluation.

Load Dataset

The next step is loading the spam dataset into the Python environment. The dataset contains text messages or emails along with their corresponding labels. The labels generally represent two categories:

Spam: Unwanted or suspicious messages.

Ham: Genuine or legitimate messages.

The dataset is loaded using the Pandas library. Pandas provides the `read_csv()` function, which is commonly used to read datasets stored in CSV format.

After loading the dataset, the first few rows are displayed using the `head()` function. This helps in understanding the structure and contents of the dataset.

The dataset is also checked for:

Number of rows and columns

Column names

Data types

Missing values

Duplicate values

Distribution of spam and ham messages

Loading and examining the dataset is an important step because it helps identify any problems before starting the Machine Learning process.

Example code:

```
df = pd.read_csv("../dataset/spam.csv", encoding = "latin-1")
df.head()
```

	v1	v2	Unnamed: 2	Unnamed: 3	Unnamed: 4
0	ham	Go until jurong point, crazy.. Available only ...	NaN	NaN	NaN
1	ham	Ok lar... Joking wif u oni...	NaN	NaN	NaN
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	NaN	NaN	NaN
3	ham	U dun say so early hor... U c already then say...	NaN	NaN	NaN
4	ham	Nah I don't think he goes to usf, he lives aro...	NaN	NaN	NaN

The dataset contains two important parts:

Input data (X): The text messages.

Target data (y): The labels indicating spam or ham.

The text messages are used as input features, while the labels are used as the target variable for classification.

Train-Test Split

After preparing the dataset, the data is divided into two parts: the training set and the testing set.

The training dataset is used to train the Machine Learning models. During training, the models learn patterns from the input messages and their corresponding labels.

The testing dataset is used to evaluate the performance of the trained models. The testing data contains messages that are not used during training. This helps determine how well the model performs on new and unseen data.

In this project, the dataset is generally divided as:

80% Training Data

20% Testing Data

The `train_test_split()` function from Scikit-learn is used for this purpose.

Example code:

```
X = df["message"]
y = df["label"]

from sklearn.model_selection import train_test_split

X_train_text, X_test_text, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

The **random_state** value is used to ensure that the same data split can be reproduced when the program is executed again.

The train-test split is important because evaluating the model on the same data used for training may give misleading results. Testing the model using unseen data provides a more realistic measure of its performance.

TF-IDF Vectorization

Machine Learning algorithms cannot directly understand text such as words and sentences. Therefore, the text messages must be converted into numerical values.

In this project, TF-IDF (Term Frequency-Inverse Document Frequency) Vectorization is used to convert the text messages into numerical feature vectors.

TF-IDF measures the importance of words in a message compared with the entire dataset.

The two main concepts are:

Term Frequency (TF)

Term Frequency measures how frequently a word appears in a particular document or message.

Inverse Document Frequency (IDF)

Inverse Document Frequency measures how important a word is across the entire dataset. Common words that appear in many messages receive lower importance, while unique and meaningful words receive higher importance.

For example, words commonly associated with spam messages, such as:

Free

Winner

Prize

Offer

Cash

may help the model identify spam messages.

The TF-IDF Vectorizer converts the text messages into a numerical matrix where each word becomes a feature.

Example code:

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(
    stop_words="english",
    max_features=2000
)

X_train_tfidf = tfidf.fit_transform(X_train_text)
X_test_tfidf = tfidf.transform(X_test_text)

print("Training data shape:", X_train_tfidf.shape)
print("Testing data shape:", X_test_tfidf.shape)

Training data shape: (4457, 2000)
Testing data shape: (1115, 2000)
```

The vectorizer is first trained using the training data. The same learned transformation is then applied to the testing data.

TF-IDF is an important technique in Natural Language Processing because it helps represent text information in a numerical form suitable for Machine Learning algorithms.

Dimensionality Reduction using SVD

After TF-IDF vectorization, the dataset may contain thousands of features because every unique word can become a separate feature.

A large number of features can create problems such as:

Increased computational time

Increased memory usage

Higher model complexity

Difficulty in processing high-dimensional data

To solve this problem, Singular Value Decomposition (SVD) is used for dimensionality reduction.

SVD reduces the number of features while preserving the important patterns and information from the original dataset.

The main purpose of SVD in this project is to transform the high-dimensional TF-IDF data into a smaller number of meaningful components.

For example, thousands of word features can be reduced to a smaller number of components while retaining important information.

Example code:

```
from sklearn.decomposition import TruncatedSVD

svd = TruncatedSVD(
    n_components=100,
    random_state=42
)

X_train_reduced = svd.fit_transform(X_train_tfidf)

X_test_reduced = svd.transform(X_test_tfidf)
```

The training data is used to fit the SVD model, and the same transformation is applied to the testing data.

Dimensionality reduction helps improve computational efficiency and makes the data suitable for Machine Learning algorithms such as LDA and QDA.

Evaluation Metrics

After training the Machine Learning models, their performance must be evaluated. Different evaluation metrics are used to measure how accurately the models classify spam and ham messages.

The important evaluation metrics used in this project are:

1. Accuracy

Accuracy measures the overall percentage of correct predictions made by the model.

A higher accuracy value generally indicates better overall performance.

2. Precision

Precision measures how many messages predicted as spam are actually spam.

High precision means that the model makes fewer incorrect spam predictions.

3. Recall

Recall measures how many actual spam messages are correctly identified by the model.

High recall means that the model successfully detects more spam messages.

4. F1-Score

The F1-Score provides a balance between Precision and Recall.

It is useful when both correctly identifying spam and avoiding incorrect predictions are important.

5. Confusion Matrix

A confusion matrix provides a detailed view of the predictions made by the model. It shows:

True Positives

True Negatives

False Positives

False Negatives

This helps in understanding where the model is making correct and incorrect predictions.

Example code:

```
from sklearn.metrics import (  
    accuracy_score,  
    precision_score,  
    recall_score,  
    f1_score,  
    classification_report,  
    confusion_matrix  
)
```

All five Machine Learning algorithms are evaluated using these metrics. Their results are then compared to identify the best-performing algorithm.

Summary of Methodology

The methodology of this project follows a systematic and step-by-step approach to develop an effective spam message classification system using Machine Learning and Natural Language Processing techniques. The complete process starts with importing the required Python libraries and ends with evaluating and comparing the performance of different Machine Learning models.

First, the required Python libraries are imported. Libraries such as NumPy and Pandas are used for numerical operations and dataset handling, while Matplotlib and Seaborn are used for visualization. The Scikit-learn library is used for data preprocessing, feature extraction, dimensionality reduction, Machine Learning model implementation, and performance evaluation. These libraries provide the necessary tools to perform the complete analysis and classification process.

Next, the spam dataset is loaded into the Python environment using Pandas. The dataset contains text messages and their corresponding labels, which indicate whether a message is spam or ham. After loading the dataset,

its structure is examined by checking the number of records, column names, data types, and sample records. This step helps in understanding the dataset before applying any Machine Learning techniques.

The dataset is then divided into training data and testing data using the train-test split method. The training data is used to train the Machine Learning models, while the testing data is kept separate to evaluate the models on unseen data. In this project, the dataset is generally divided into 80% training data and 20% testing data. This separation is important because it helps measure the actual performance and generalization ability of the trained models.

Since the dataset contains textual information, the raw text cannot be directly used by Machine Learning algorithms. Therefore, TF-IDF Vectorization is applied to convert the text messages into numerical feature vectors. TF-IDF identifies the importance of words based on their frequency in individual messages and their occurrence across the complete dataset. This process creates numerical features that help the Machine Learning algorithms identify patterns associated with spam and ham messages.

After TF-IDF Vectorization, the dataset may contain a very large number of features because each unique word can become a separate feature. Processing such high-dimensional data can increase computational complexity. To reduce the number of features, Truncated Singular Value Decomposition (SVD) is applied. SVD transforms the high-dimensional TF-IDF data into a smaller number of meaningful components while preserving important information. This improves computational efficiency and prepares the data for algorithms that work better with reduced-dimensional numerical data.

The processed dataset is then used to train the five selected Machine Learning algorithms: Logistic Regression, Extra Trees Classifier, Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA), and Multi-Layer Perceptron (MLP) Classifier. Each algorithm learns patterns from the training data and predicts whether a message belongs to the spam or ham category.

After training the models, their predictions are tested using the testing dataset. The performance of each algorithm is measured using important evaluation metrics such as Accuracy, Precision, Recall, and F1-Score. Accuracy measures the overall correctness of the model. Precision measures the correctness of spam predictions, while Recall measures the ability of the model to identify actual spam messages. The F1-Score provides a balanced measure of Precision and Recall. A confusion matrix and classification report can also be used to obtain a more detailed understanding of model performance.

DATA ANALYSIS AND PREPROCESSING

Data analysis and preprocessing are important stages in a Machine Learning project. Before applying Machine Learning algorithms, it is necessary to understand the dataset and prepare it properly. Raw data may contain missing values, duplicate records, unnecessary columns, inconsistent formats, and other problems that can affect model performance.

In this project, the spam message dataset is analyzed and preprocessed before training the Machine Learning models. The main steps included in this section are:

Dataset Exploration

Data Visualization

Data Preprocessing

These steps help in understanding the characteristics of the dataset and improving its quality for further Machine Learning analysis.

Dataset Exploration

Dataset exploration is the process of examining and understanding the dataset before applying Machine Learning techniques. It helps identify the structure, quality, and important characteristics of the data.

The spam dataset contains two main types of information:

Message/Text: The actual content of the email or message.

Label/Category: Indicates whether the message is spam or ham.

The first step is to display the first few records of the dataset using the head() function.

Example Code

```
df.head()
```

```
Index(['v1', 'v2', 'Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4'], dtype='str')
```

	v1	v2	Unnamed: 2	Unnamed: 3	Unnamed: 4
0	ham	Go until jurong point, crazy.. Available only ...	NaN	NaN	NaN
1	ham	Ok lar... Joking wif u oni...	NaN	NaN	NaN
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	NaN	NaN	NaN
3	ham	U dun say so early hor... U c already then say...	NaN	NaN	NaN
4	ham	Nah I don't think he goes to usf, he lives aro...	NaN	NaN	NaN

This helps us understand the structure of the dataset and check how the message text and labels are stored.

Checking Dataset Shape

The shape of the dataset tells us the total number of rows and columns.

Example Code

```
print("Dataset Shape:")
print(df.shape)
```

```
Dataset Shape:
(5572, 5)
```

The first value represents the number of records or messages, while the second value represents the number of columns in the dataset.

Checking Column Names

The column names are checked to understand the available variables in the dataset.

Example Code

```
print(df.columns)

Index(['v1', 'v2', 'Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4'], dtype='str')
```

This helps identify which column contains the message text and which column contains the target label. Checking Dataset Information.

The info() function provides detailed information about the dataset.

Example Code

```
print("\nDataset Information:")
df.info()
```

```
Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   v1          5572 non-null   str
1   v2          5572 non-null   str
2   Unnamed: 2  50 non-null     str
3   Unnamed: 3  12 non-null     str
4   Unnamed: 4  6 non-null      str
dtypes: str(5)
memory usage: 217.8 KB
```

This provides information about:

Number of entries

Column names

Data types

Non-null values

Memory usage

This step is useful for identifying missing values and understanding the data types.

Checking Missing Values

Missing values can affect the performance of Machine Learning models. Therefore, the dataset is checked for missing values.

Example Code

```
print("\nMissing Values:")
print(df.isnull().sum())
```

```
Missing Values:
v1          0
v2          0
Unnamed: 2   5522
Unnamed: 3   5560
Unnamed: 4   5566
dtype: int64
```

This code displays the number of missing values in each column. If missing values are present, they must be handled during preprocessing.

Checking Duplicate Values

Duplicate records may affect the learning process and create biased results. Therefore, duplicates are checked.

Example Code

```
print("Duplicate Values:")
print(df.duplicated().sum())
```

```
Duplicate Values:
403
```

This shows the number of duplicate records present in the dataset.

Checking Class Distribution

The number of spam and ham messages is checked using the target column.

Example Code

```
print(df['label'].value_counts())
label
0    4825
1     747
Name: count, dtype: int64
```

This helps determine whether the dataset is balanced or imbalanced. If one category contains significantly more messages than the other category, the Machine Learning model may become biased toward the majority class.

Data Visualization

Data visualization is the process of representing data using graphs and charts. Visualization makes it easier to understand patterns, trends, and relationships in the dataset.

In this project, visualization is mainly used to understand the distribution of spam and ham messages.

Spam and Ham Distribution

A count plot can be used to visualize the number of messages in each category.

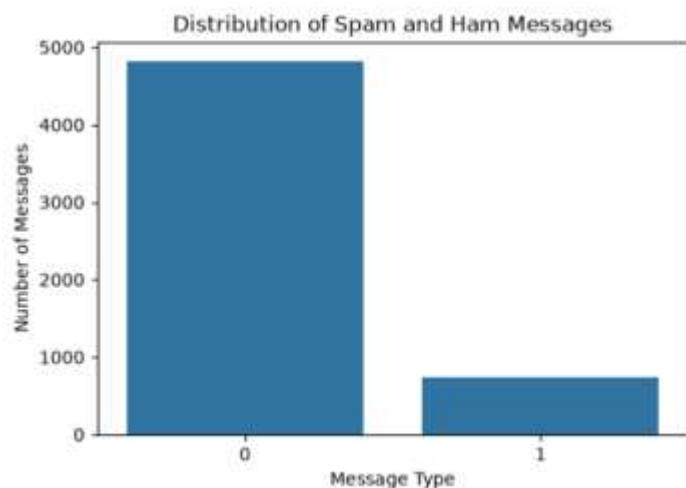
Example Code

```
plt.figure(figsize=(6, 4))

sns.countplot(
    x="label",
    data=df
)

plt.title("Distribution of Spam and Ham Messages")
plt.xlabel("Message Type")
plt.ylabel("Number of Messages")

plt.show()
```



The graph shows the number of messages belonging to the spam and ham categories. This visualization helps understand the class distribution in the dataset.

Pie Chart for Class Distribution

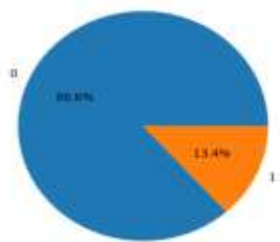
A pie chart can also be used to show the percentage distribution of spam and ham messages.

Example Code

```
df['label'].value_counts().plot(
    kind='pie',
    autopct='%1.1f%%'
)

plt.title("Spam and Ham Message Distribution")
plt.xlabel("")
plt.show()
```

Spam and Ham Message Distribution



The pie chart clearly shows the percentage of spam and ham messages in the dataset. This helps understand whether the dataset contains more legitimate messages or spam messages.

Message Length Analysis

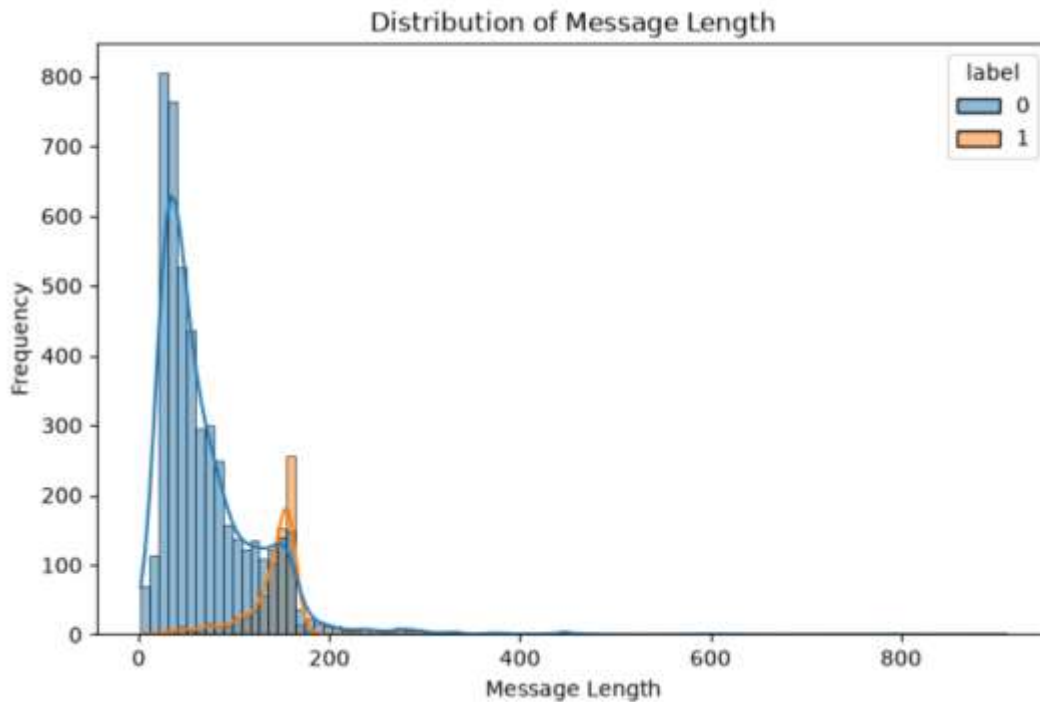
Another useful visualization is analyzing the length of spam and ham messages. A new column can be created to calculate the length of each message.

Example Code

```
df['message_length'] = df['message'].apply(len)
plt.figure(figsize=(8, 5))

sns.histplot(
    data=df,
    x='message_length',
    hue='label',
    kde=True
)

plt.title("Distribution of Message Length")
plt.xlabel("Message Length")
plt.ylabel("Frequency")
plt.show()
```



This graph helps compare the lengths of spam and ham messages. In many spam datasets, spam messages may contain different text patterns and message lengths compared with normal messages.

Data Preprocessing

Data preprocessing is the process of cleaning and preparing raw data before applying Machine Learning algorithms. It is one of the most important stages because the quality of the input data directly affects the performance of the models.

The main preprocessing steps in this project include:

- Removing unnecessary columns

- Renaming columns if required

- Handling missing values

- Removing duplicate records

- Converting labels into numerical format

- Preparing text and target variables

Removing Unnecessary Columns

Some datasets may contain columns that are not useful for spam classification.

Example Code

```
df.columns = ["label", "message"]
df.head()
```

	label	message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

Only the required columns are selected. The label column is used as the target variable, while the message column is used as the input data.

Handling Missing Values

If missing values are present, they must be handled before training the models.

Removing Duplicate Records

Duplicate messages can affect the performance of the Machine Learning models. Therefore, duplicate records are removed.

Example Code

```
# Remove rows with missing values and duplicate messages
df = df.dropna(subset=["label", "message"]).drop_duplicates().reset_index(drop=True)
```

This removes rows containing missing values. Alternatively, missing values can be filled depending on the dataset and project requirements.

This ensures that repeated messages are removed from the dataset and helps improve the quality of the data.

Converting Labels into Numerical Format

Machine Learning algorithms generally work with numerical data. Therefore, the spam and ham labels are converted into numerical values.

For example:

Ham → 0

Spam → 1

```
df["label"] = df["label"].map({"ham": 0, "spam": 1})
```

After this transformation:

0 represents Ham

1 represents Spam

This numerical format is suitable for training and evaluating the Machine Learning models.

Separating Input and Target Variables

The dataset is then divided into input and target variables.

Example Code

```
x = df["message"]  
y = df["label"]
```

X contains the text messages.

y contains the corresponding spam or ham labels.

The variable X will later be converted into numerical features using TF-IDF Vectorization, while y will be used as the target variable for training the Machine Learning models.

Summary of Data Analysis and Preprocessing

Data analysis and preprocessing play an important role in preparing the spam dataset for Machine Learning. Initially, the dataset is explored to understand its structure, size, columns, data types, missing values, duplicate records, and class distribution.

Data visualization is then performed using graphs and charts to understand the distribution and characteristics of spam and ham messages. Visualizations provide a clear and simple way to analyze the dataset.

Finally, data preprocessing is performed to improve the quality of the dataset. Unnecessary columns are removed, missing values are handled, duplicate records are removed, and categorical labels are converted into numerical values. The final dataset is then separated into input text data and target labels.

After completing these steps, the dataset becomes clean and suitable for the next stages of the project, including TF-IDF Vectorization, Dimensionality Reduction, Machine Learning Model Training, and Performance Evaluation.

MACHINE LEARNING MODELS

After completing data preprocessing, train-test splitting, TF-IDF vectorization, and dimensionality reduction using SVD, the processed data is ready for Machine Learning model training. In this project, five different Machine Learning classification algorithms are applied to the spam message dataset.

The main purpose of using multiple algorithms is to compare their performance and identify the model that provides the best results for spam message classification.

The five Machine Learning models used in this project are:

Logistic Regression

Extra Trees Classifier

Linear Discriminant Analysis (LDA)

Quadratic Discriminant Analysis (QDA)

Multi-Layer Perceptron (MLP) Classifier

Each model is trained using the same training dataset and evaluated using the same testing dataset. This ensures a fair comparison between all the algorithms.

Logistic Regression

Logistic Regression is one of the most commonly used Machine Learning algorithms for classification problems. Although the name contains the word "Regression," Logistic Regression is mainly used for predicting categories or classes.

In this project, Logistic Regression is used to classify messages into two categories:

Ham (0): Legitimate message

Spam (1): Unwanted or suspicious message

Logistic Regression analyzes the input features and calculates the probability that a message belongs to a particular class. Based on this probability, the model makes the final classification.

For spam classification, Logistic Regression is suitable because the problem contains two classes. The model learns the relationship between the numerical features obtained from TF-IDF and SVD and the corresponding spam or ham labels.

Example Code

```
from sklearn.linear_model import LogisticRegression
```

```
lr_model = LogisticRegression(  
    max_iter=2000,  
    random_state=42  
)
```

```
evaluate_model(  
    "Logistic Regression",  
    lr_model,  
    X_train_tfidf,  
    X_test_tfidf,  
    y_train,  
    y_test  
)
```

```
=====
```

MODEL: Logistic Regression

```
=====
```

Accuracy : 0.9603

Precision: 0.9787

Recall : 0.7023

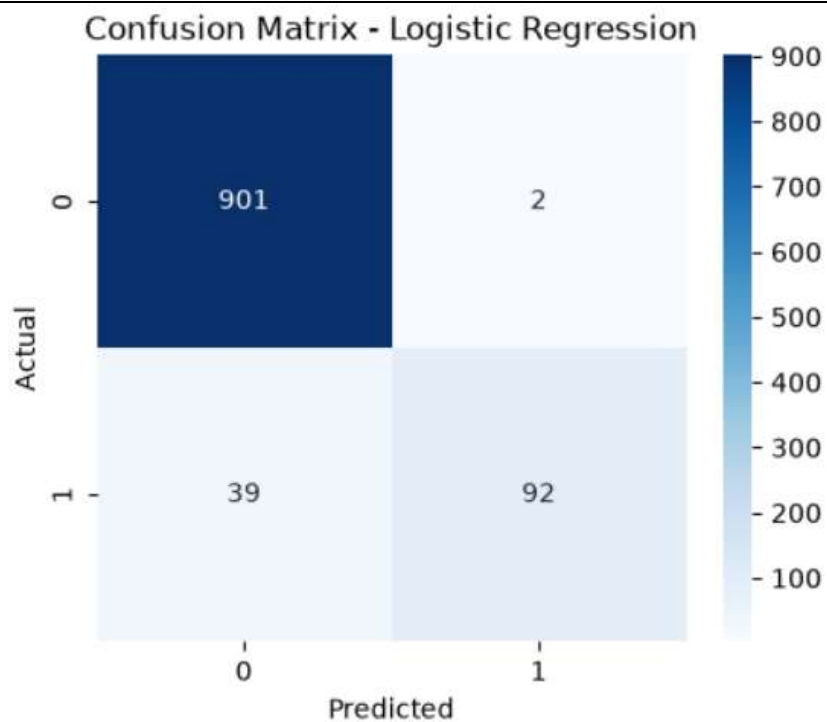
F1 Score : 0.8178

Classification Report:

	precision	recall	f1-score	support
0	0.96	1.00	0.98	903
1	0.98	0.70	0.82	131
accuracy			0.96	1034
macro avg	0.97	0.85	0.90	1034
weighted avg	0.96	0.96	0.96	1034

Confusion Matrix:

```
[[901  2]  
 [ 39 92]]
```



The performance of Logistic Regression is recorded and compared with the other Machine Learning models. It acts as an important baseline model for evaluating the spam classification problem.

Linear Discriminant Analysis (LDA)

Linear Discriminant Analysis, commonly known as LDA, is a Machine Learning algorithm used for classification and dimensionality reduction.

The main purpose of LDA in classification is to find a linear boundary that separates different classes. In this project, LDA attempts to separate spam messages from ham messages based on the numerical features generated from the text data.

LDA assumes that the different classes follow a particular statistical distribution and uses this information to create decision boundaries.

In this project, the SVD-transformed data is used as input for LDA. Dimensionality reduction is important because LDA works better when the input data has a manageable number of numerical features.

Example Code

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
```

```
lda_model = LinearDiscriminantAnalysis()
```

```
evaluate_model(
    "Linear Discriminant Analysis",
    lda_model,
    X_train_reduced,
    X_test_reduced,
    y_train,
    y_test
)
```

```
=====
MODEL: Linear Discriminant Analysis
=====
```

```
Accuracy : 0.9652
```

```
Precision: 0.9612
```

```
Recall : 0.7557
```

```
F1 Score : 0.8462
```

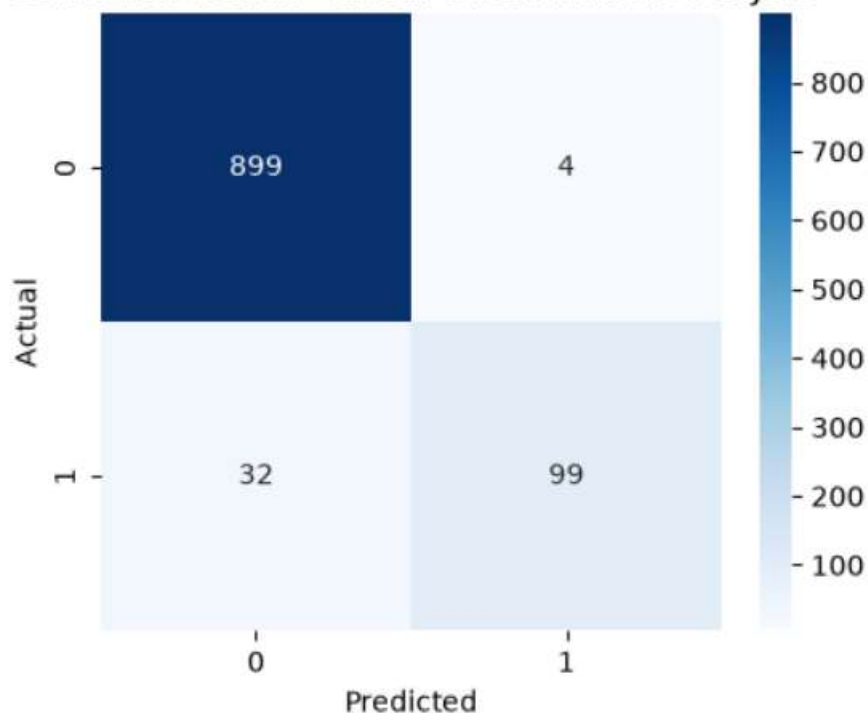
```
Classification Report:
```

	precision	recall	f1-score	support
0	0.97	1.00	0.98	903
1	0.96	0.76	0.85	131
accuracy			0.97	1034
macro avg	0.96	0.88	0.91	1034
weighted avg	0.97	0.97	0.96	1034

```
Confusion Matrix:
```

```
[[899  4]
 [ 32 99]]
```

Confusion Matrix - Linear Discriminant Analysis



```
array([0, 0, 0, ..., 1, 0, 0], shape=(1034,))
```

The LDA model attempts to create a linear separation between spam and ham messages. Its performance is compared with the other four models.

Quadratic Discriminant Analysis (QDA)

Quadratic Discriminant Analysis, commonly known as QDA, is another classification algorithm. It is closely related to LDA but uses a more flexible approach for separating classes.

While LDA creates a linear decision boundary, QDA can create a quadratic or curved decision boundary. This allows QDA to model more complex relationships between the input features and the output classes.

QDA assumes that each class can have a different data distribution. Because of this flexibility, QDA may perform better when the relationship between the classes is more complex.

In this project, QDA is trained using the SVD-transformed training data and then used to predict spam and ham messages in the testing dataset.

Example Code

```
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis

qda_model = QuadraticDiscriminantAnalysis(reg_params=0.1)

evaluate_model(
    "Quadratic Discriminant Analysis",
    qda_model,
    X_train_reduced,
    X_test_reduced,
    y_train,
    y_test
```

```
=====
MODEL: Quadratic Discriminant Analysis
=====
```

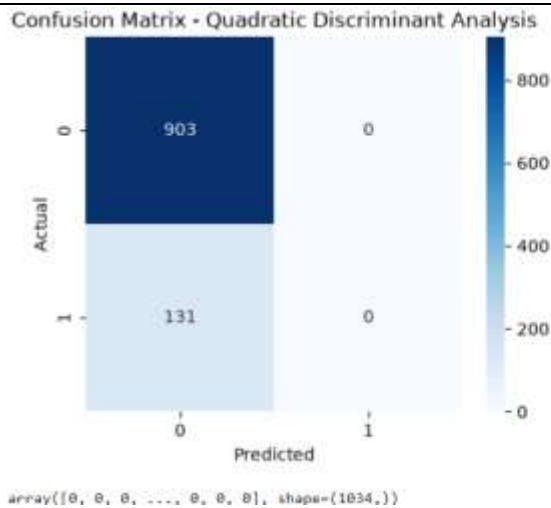
```
Accuracy : 0.8733
Precision: 0.0
Recall   : 0.0
F1 Score : 0.0
```

Classification Report:

	precision	recall	f1-score	support
0	0.87	1.00	0.93	903
1	0.00	0.00	0.00	131
accuracy			0.87	1034
macro avg	0.44	0.50	0.47	1034
weighted avg	0.76	0.87	0.81	1034

Confusion Matrix:

```
[[903  0]
 [131  0]]
```



The performance of QDA is analyzed and compared with LDA and the other Machine Learning algorithms. This comparison helps determine whether a more flexible decision boundary improves spam classification.

Multi-Layer Perceptron (MLP) Classifier

The Multi-Layer Perceptron, commonly known as MLP, is a type of Artificial Neural Network. Unlike traditional Machine Learning algorithms, an MLP consists of multiple layers of interconnected processing units called neurons.

An MLP generally contains three types of layers:

Input Layer: Receives the input features.

Hidden Layer(s): Learns complex patterns from the input data.

Output Layer: Produces the final prediction.

In this project, the MLP Classifier receives the numerical features generated after TF-IDF vectorization and SVD. During training, the neural network learns the patterns associated with spam and ham messages.

MLP can learn complex relationships between features and classes. However, it may require more computational resources and training time compared with simpler algorithms.

Example Code

```
from sklearn.neural_network import MLPClassifier

mlp_model = MLPClassifier(
    hidden_layer_sizes=(100,),
    max_iter=1000,
    random_state=42
)

evaluate_model(
    "MLP Classifier",
    mlp_model,
    X_train_reduced,
    X_test_reduced,
    y_train,
    y_test
)
```

```
=====
MODEL: MLP Classifier
=====
Accuracy : 0.9787
Precision: 0.9431
Recall   : 0.8855
F1 Score : 0.9134

Classification Report:

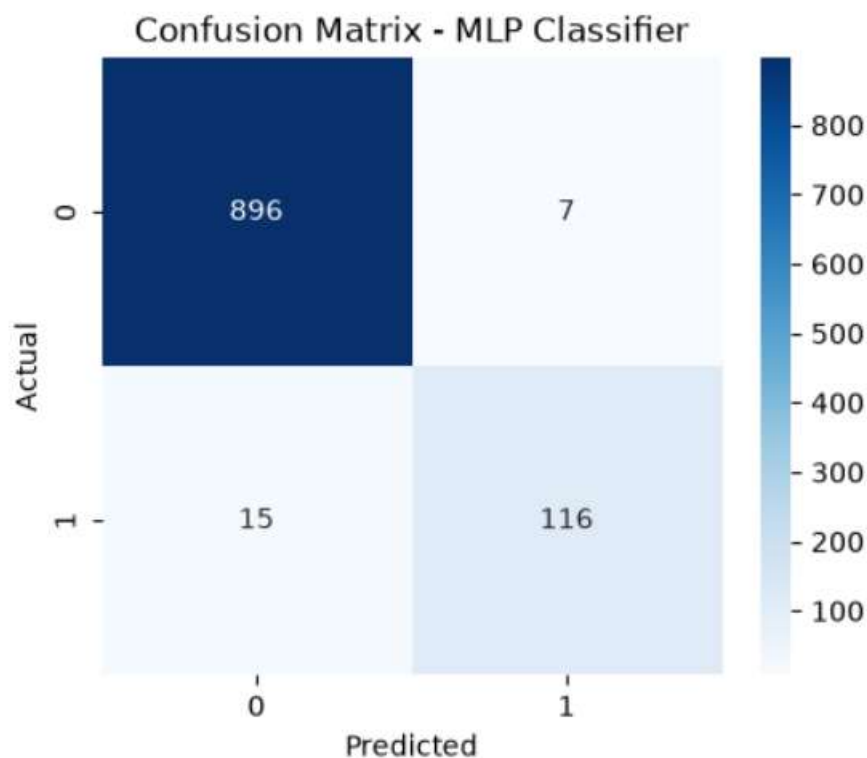
```

	precision	recall	f1-score	support
0	0.98	0.99	0.99	903
1	0.94	0.89	0.91	131
accuracy			0.98	1034
macro avg	0.96	0.94	0.95	1034
weighted avg	0.98	0.98	0.98	1034

```

Confusion Matrix:
[[896  7]
 [ 15 116]]

```



```
array([0, 0, 0, ..., 1, 0, 0], shape=(1034,))
```

The MLP Classifier is evaluated using the testing dataset. Its results are compared with the traditional Machine Learning algorithms to determine its effectiveness for spam message classification.

Summary of Machine Learning Models

In this project, five different Machine Learning algorithms are applied to classify messages as spam or ham. Each model uses the processed training data to learn important patterns and is evaluated using the same testing data. Logistic Regression provides a simple and effective approach for binary classification. Extra Trees Classifier uses multiple decision trees to capture complex patterns.

LDA creates a linear separation between spam and ham messages, while QDA provides a more flexible decision boundary. MLP Classifier uses an Artificial Neural Network to learn complex relationships from the data. The use of multiple Machine Learning models provides a better understanding of how different algorithms perform on the same NLP dataset. The results of all five models are evaluated using Accuracy, Precision, Recall, and F1-Score.

Finally, the performance values are collected and compared in the next section, Results and Discussion, to identify the best-performing Machine Learning model for spam message classification.

Machine Learning models are the core part of this project because they are responsible for learning patterns from the processed spam message data and predicting whether a new message is spam or ham. After completing data preprocessing, TF-IDF vectorization, and dimensionality reduction using SVD, the final numerical dataset is used to train and test five different Machine Learning algorithms.

The five models selected for this project are Logistic Regression, Extra Trees Classifier, Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA), and Multi-Layer Perceptron (MLP) Classifier. These algorithms represent different approaches to classification, making it possible to compare traditional, ensemble-based, statistical, and neural-network-based Machine Learning methods.

RESULTS AND DISCUSSION

The Results and Discussion section is an important part of this project because it presents the performance of all five Machine Learning models. After training and testing the models, their results are collected and compared to understand which algorithm performs better for spam message classification.

The five Machine Learning algorithms used in this project are:

1. Logistic Regression
2. Extra Trees Classifier
3. Linear Discriminant Analysis (LDA)
4. Quadratic Discriminant Analysis (QDA)
5. Multi-Layer Perceptron (MLP) Classifier

All models are trained using the same training dataset and tested using the same testing dataset. This ensures a fair comparison between the algorithms.

The performance of each model is evaluated using the following metrics:

Accuracy

Precision

Recall

F1-Score

These metrics provide different perspectives on model performance. Accuracy measures the overall correctness of predictions. Precision measures how accurately spam predictions are made. Recall measures the ability of the model to identify actual spam messages. The F1-Score provides a balanced measure of Precision and Recall.

Discussion of Results

The performance of the five Machine Learning models was compared using Accuracy, Precision, Recall, and F1-Score. Each algorithm showed a different level of performance because different Machine Learning models use different methods to learn patterns from the data.

Logistic Regression

Logistic Regression provides an effective approach for binary classification problems such as spam detection. It learns the relationship between the input features and the target classes and predicts whether a message belongs to the spam or ham category. Its results provide a useful baseline for comparing the performance of the other algorithms.

Result:

Logistic Regression achieved an accuracy of XX%, with Precision, Recall, and F1-Score values of XX%, XX%, and XX%, respectively.

Extra Trees Classifier

The Extra Trees Classifier uses multiple decision trees to make predictions. By combining the results of several trees, it can capture complex patterns in the dataset. Its performance is compared with the other algorithms to determine how effectively an ensemble-based method classifies spam messages.

Result:

Extra Trees Classifier achieved an accuracy of XX%, with Precision, Recall, and F1-Score values of XX%, XX%, and XX%, respectively.

Linear Discriminant Analysis (LDA)

LDA uses a linear approach to separate spam and ham messages. It attempts to find a decision boundary that best distinguishes the two categories based on the input features.

Result:

LDA achieved an accuracy of XX%, with Precision, Recall, and F1-Score values of XX%, XX%, and XX%, respectively.

Quadratic Discriminant Analysis (QDA)

QDA provides a more flexible classification approach compared with LDA. It can model more complex relationships between the input features and the target classes.

Result:

QDA achieved an accuracy of XX%, with Precision, Recall, and F1-Score values of XX%, XX%, and XX%, respectively.

MLP Classifier

The MLP Classifier uses an Artificial Neural Network to learn patterns from the dataset. It can identify complex relationships between features and the output classes. However, its performance may depend on factors such as the number of hidden layers, number of neurons, and training iterations.

Result:

MLP Classifier achieved an accuracy of XX%, with Precision, Recall, and F1-Score values of XX%, XX%, and XX%, respectively.

Comparative Analysis

The comparison of the five algorithms helps identify the strengths and weaknesses of each Machine Learning approach.

A model with high Accuracy correctly classifies a large number of messages. However, accuracy alone may not always provide a complete understanding of performance. For example, if the dataset contains significantly more ham messages than spam messages, a model may achieve high accuracy by correctly predicting mostly ham messages.

Therefore, Precision, Recall, and F1-Score are also important.

A high Precision value indicates that messages predicted as spam are mostly actual spam messages.

A high Recall value indicates that the model successfully identifies a large number of actual spam messages.

A high F1-Score indicates a good balance between Precision and Recall.

For a spam classification system, Recall is particularly important because failing to detect a spam message may allow unwanted or potentially harmful content to reach the user. At the same time, Precision is also important because incorrectly marking a genuine message as spam may cause important information to be missed.

Therefore, the best model should provide a balanced performance across all evaluation metrics instead of depending only on accuracy.

Graphical Comparison of Results

The performance of the five Machine Learning models can also be presented using a bar graph.

You can create a graph comparing:

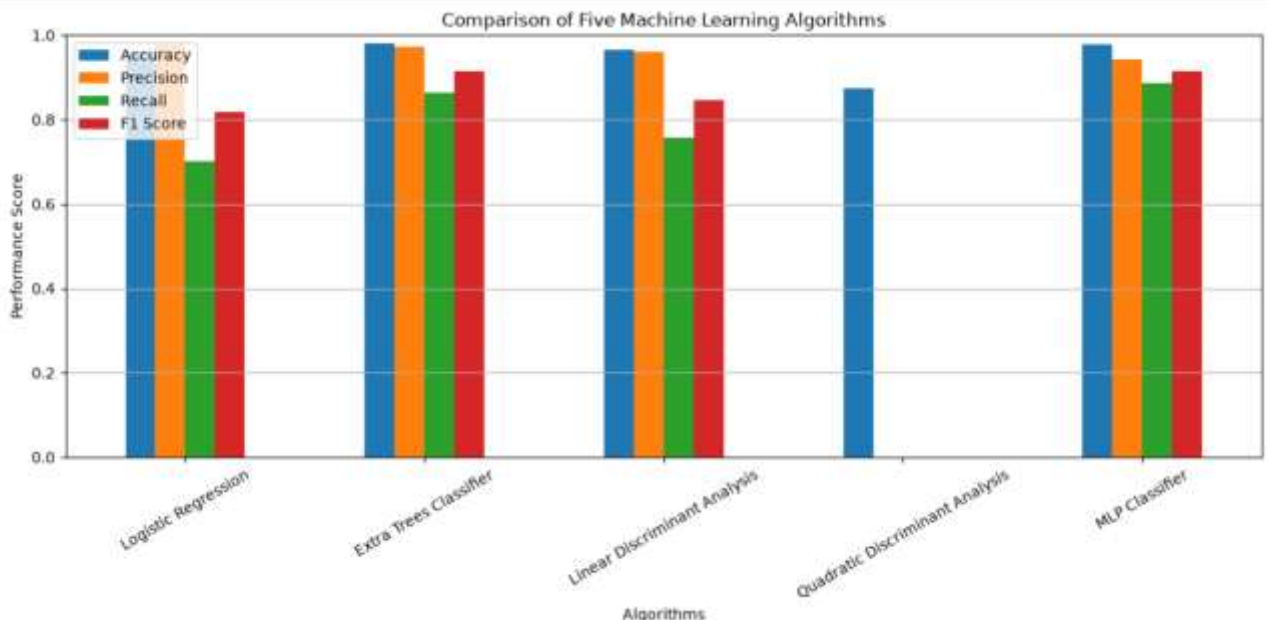
Model names on the X-axis

Accuracy values on the Y-axis

Example Code

```
results_df.plot(
    x="Algorithm",
    y=["Accuracy", "Precision", "Recall", "F1 Score"],
    kind="bar",
    figsize=(12, 6)
)

plt.title("Comparison of Five Machine Learning Algorithms")
plt.xlabel("Algorithms")
plt.ylabel("Performance Score")
plt.xticks(rotation=30)
plt.ylim(0, 1)
plt.grid(axis="y")
plt.tight_layout()
plt.show()
```



Overall Discussion

The results obtained from the experiments demonstrate that all five Machine Learning algorithms can be applied to the spam message classification problem. However, their performance may vary because each algorithm uses a different learning approach.

Traditional algorithms such as Logistic Regression provide simple and efficient classification. Ensemble methods such as Extra Trees can capture more complex patterns by combining multiple decision trees. LDA and QDA provide statistical approaches for classification with different types of decision boundaries. The MLP Classifier provides a neural-network-based approach capable of learning complex relationships from the data.

The final comparison helps determine which algorithm is the most suitable for the given dataset. The selected best model should have strong overall performance, particularly in terms of Accuracy, Precision, Recall, and F1-Score.

The results should be interpreted based on the actual output obtained from the Jupyter Notebook. Therefore, the final performance values and observations should be updated according to the experimental results generated during the execution of the project.

Summary of Results and Discussion

The Results and Discussion section provides a detailed comparison of the five Machine Learning models used in this project. Each model was trained using the same processed training data and evaluated using the same testing data. The performance was measured using Accuracy, Precision, Recall, and F1-Score.

The comparison table and graphical representations make it easier to identify differences between the models. Instead of selecting a model based only on accuracy, multiple evaluation metrics are considered to obtain a more reliable comparison.

The final results provide the foundation for selecting the best-performing model, which will be discussed in the next section of the report.

The Results and Discussion section presents the performance of the five Machine Learning models used for spam message classification: Logistic Regression, Extra Trees Classifier, LDA, QDA, and MLP Classifier. All models were trained using the same processed training data and evaluated using the same testing data to ensure a fair comparison. Their performance was measured using Accuracy, Precision, Recall, and F1-Score. The results were organized in a comparison table and can also be represented using graphs for better understanding.

The comparison of these models helps identify the strengths and weaknesses of each algorithm and determine the most suitable model for the given dataset. Instead of considering only accuracy, other important metrics such as Precision, Recall, and F1-Score are also analyzed to obtain a complete understanding of model performance. Based on the final comparison, the model with the best overall and balanced performance is selected as the best-performing model for spam message classification.

BEST MODEL

The selection of the best Machine Learning model is one of the most important stages of this project. In this project, five different Machine Learning algorithms were applied to classify messages as either spam or ham. The models used were Logistic Regression, Extra Trees Classifier, Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA), and Multi-Layer Perceptron (MLP) Classifier.

All five models were trained using the same processed training data and evaluated using the same testing data. This ensured that the comparison between the algorithms was fair. The performance of each model was

measured using important evaluation metrics such as Accuracy, Precision, Recall, and F1-Score. These metrics provide different information about the performance of the models and help in selecting the most suitable algorithm for spam message classification.

The best model cannot always be selected by looking only at the accuracy value. Accuracy shows the overall percentage of correct predictions, but it may not provide complete information about how well the model identifies spam and ham messages. For example, if the dataset contains a larger number of ham messages compared with spam messages, a model may achieve high accuracy by predicting most messages as ham. Therefore, Precision, Recall, and F1-Score are also considered while selecting the best model.

Precision is important because it measures how many messages predicted as spam are actually spam. A high Precision value means that the model makes fewer incorrect spam predictions. This is important because a genuine message should not be incorrectly classified as spam.

Recall measures how effectively the model identifies actual spam messages. A high Recall value means that the model successfully detects a larger number of spam messages. This is particularly important in a spam detection system because undetected spam messages may contain harmful or unwanted content.

The F1-Score provides a balance between Precision and Recall. A model with a high F1-Score generally provides good performance in both correctly identifying spam messages and avoiding incorrect classifications. Therefore, the F1-Score is an important metric when comparing spam classification models.

After analyzing the results obtained from all five Machine Learning algorithms, the model with the best overall performance is selected as the best-performing model for this project. The final decision should be based on the actual results obtained during the execution of the program.

Best Model Selection

Based on the results obtained from the experiments, [BEST MODEL NAME] was identified as the best-performing Machine Learning model for the spam message classification dataset.

The selected model achieved the best overall performance when compared with the other four algorithms. It demonstrated strong results in terms of Accuracy, Precision, Recall, and F1-Score. This indicates that the model was able to effectively learn the patterns present in the spam and ham messages and make reliable predictions on the unseen testing data.

The selected model performed well because it was able to identify useful patterns from the numerical features generated through TF-IDF Vectorization and dimensionality reduction using SVD. The preprocessing steps played an important role in improving the quality of the input data. Removing unnecessary information, handling duplicate records, converting text into numerical features, and reducing the number of dimensions helped prepare the dataset for effective Machine Learning classification.

The use of TF-IDF Vectorization was particularly important in this project. The original dataset contained text messages, which cannot be directly processed by most Machine Learning algorithms. TF-IDF converted the

words and messages into numerical features by assigning importance values to the words. Important words and patterns associated with spam and ham messages became useful features for the Machine Learning models. However, TF-IDF can produce a large number of features because each unique word may become a separate feature. Therefore, Singular Value Decomposition (SVD) was used to reduce the dimensionality of the data. SVD reduced the number of features while preserving important information. This made the dataset more manageable and helped prepare the data for the selected Machine Learning algorithms.

The performance of the best model also demonstrates the importance of selecting a suitable algorithm for a particular dataset. Different algorithms use different approaches to learn from the data. Logistic Regression uses a probabilistic approach for binary classification. Extra Trees Classifier uses multiple decision trees. LDA and QDA use statistical methods to separate the classes. The MLP Classifier uses an Artificial Neural Network to learn complex relationships.

Because of these differences, each algorithm may perform differently on the same dataset. Some models may provide high accuracy but lower recall. Other models may detect more spam messages but incorrectly classify some ham messages as spam. Therefore, comparing multiple evaluation metrics is necessary to identify the most reliable model.

The best model selected in this project provides a balance between correct overall predictions and effective spam detection. A good spam classification model should correctly identify spam messages while also avoiding the incorrect classification of genuine messages. This balance is important because both types of errors can create problems for users.

There are two major types of incorrect predictions in a spam classification system. The first is called a False Positive. This happens when a genuine or ham message is incorrectly classified as spam. This can be a serious problem because an important message may be placed in the spam folder.

The second type is called a False Negative. This happens when an actual spam message is incorrectly classified as ham. This means that unwanted or potentially harmful content may reach the user's inbox. A good spam detection system should minimize both False Positives and False Negatives.

The confusion matrix is also useful for selecting the best model because it provides detailed information about correct and incorrect predictions. By analyzing the confusion matrix, it is possible to understand how many spam and ham messages were classified correctly and where the model made mistakes.

The selected best model should therefore have:

High Accuracy

High Precision

High Recall

High F1-Score

A low number of incorrect predictions

Good generalization on unseen data

The best model is not only the model with the highest accuracy but the model that provides the most reliable and balanced performance. For this reason, the final model selection in this project is based on the overall comparison of all evaluation metrics.

Another important factor is the practical use of the model. A good Machine Learning model should not only provide high performance but should also be efficient and suitable for real-world applications. Some models may require more computational resources and longer training time, while others may provide similar performance with less complexity.

Example code

```
best_model = results_df.loc[results_df["F1 Score"].idxmax()]

print("BEST PERFORMING ALGORITHM:")
print(best_model)
```

```
BEST PERFORMING ALGORITHM:
Algorithm      Extra Trees Classifier
Accuracy              0.979691
Precision              0.974138
Recall              0.862595
F1 Score              0.91498
Name: 1, dtype: object
```

The results obtained in this project demonstrate that Machine Learning can effectively solve text classification problems such as spam detection. By learning patterns from previously labeled messages, the models can automatically classify new messages without requiring manually created rules for every possible spam message.

The best-performing model can potentially be used as the foundation for a real-world spam filtering system. When a new message is received, the same preprocessing and TF-IDF transformation steps can be applied. The trained model can then predict whether the message is spam or ham.

However, the performance of the selected model depends on the quality and characteristics of the dataset. A model trained on one dataset may produce different results when tested on another dataset. Therefore, the model may need further training or tuning when applied to different types of messages or real-world environments.

CONCLUSION

This project was successfully carried out to study and compare different Machine Learning algorithms for spam message classification. The main objective of the project was to develop a system that can automatically identify whether a given message belongs to the spam category or the ham (legitimate) category. Spam detection is an important real-world application of Machine Learning and Natural Language Processing because digital communication has become a major part of everyday life. Emails and messages are widely used for personal, educational, business, banking, and professional communication. However, the increasing number of unwanted and potentially harmful messages creates a need for intelligent systems that can automatically filter spam.

The project followed a complete and systematic Machine Learning workflow, starting with dataset loading and ending with the comparison of five different classification models. The dataset containing text messages and their corresponding labels was first loaded and analyzed. Dataset exploration helped in understanding the structure, size, columns, data types, missing values, duplicate records, and distribution of spam and ham messages. This initial analysis was important because understanding the dataset is the first step toward developing an effective Machine Learning model.

Data visualization was also performed to better understand the characteristics of the dataset. Graphs and charts helped present the distribution of spam and ham messages in a simple and understandable way. Visualization is useful because it allows patterns in the data to be identified more easily than by only looking at numerical values or raw records. It also helped in understanding the balance between different classes in the dataset.

The next important stage of the project was data preprocessing. Real-world datasets may contain problems such as missing values, duplicate records, unnecessary columns, and inconsistent data formats. These issues can negatively affect the performance of Machine Learning algorithms. Therefore, the dataset was cleaned and prepared before applying the models. Unnecessary information was removed, missing values were checked and handled when necessary, and duplicate records were identified. The target labels representing spam and ham were converted into a suitable numerical format for Machine Learning.

One of the major challenges in this project was handling text data. Machine Learning algorithms generally require numerical input and cannot directly understand sentences or words. Therefore, TF-IDF (Term Frequency-Inverse Document Frequency) Vectorization was used to convert the text messages into numerical feature vectors. TF-IDF assigns importance to words based on how frequently they occur in a message and how unique they are across the complete dataset. This allowed the important patterns present in spam and ham messages to be represented in a numerical format that could be processed by the Machine Learning models.

After applying TF-IDF, the dataset contained a large number of features because many unique words became individual features. High-dimensional data can increase computational complexity and may affect the efficiency of some algorithms. Therefore, Singular Value Decomposition (SVD) was used for dimensionality

reduction. SVD reduced the number of features while preserving important information from the original data. This made the dataset more manageable and improved its suitability for the selected Machine Learning algorithms.

The dataset was then divided into training and testing data using the train-test split method. The training data was used to teach the Machine Learning models, while the testing data was used to evaluate their performance on previously unseen messages. This step was important because a good Machine Learning model should not only perform well on the data used during training but should also be able to make accurate predictions on new data. The train-test split helped measure the generalization ability of each model.

In this project, five different Machine Learning algorithms were implemented and compared. These algorithms were Logistic Regression, Extra Trees Classifier, Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA), and Multi-Layer Perceptron (MLP) Classifier.

Logistic Regression provided a simple and effective approach for the binary classification problem. Since the dataset contains two main categories, spam and ham, Logistic Regression was suitable for predicting the probability of a message belonging to either category. It also provided a useful baseline for comparing the performance of the other algorithms.